Solutions to Asymptotic Complexity Questions

Q1 (GATE 2021 Style)

Given $T(n) = T(n/2) + T(2n/5) + 7n$ and $T(0) = 1$. Determine the asymptotic complexity of $T(n)$.

Solution: We can analyze this using a recursion tree. The total work at the root is $7n$. The total work at the next level is $7(n/2) + 7(2n/5) = 7n(1/2 + 2/5) = 7n(9/10)$.
Since the ratio of work at each subsequent level is $9/10 < 1$, the total work is a geometrically decreasing series. The sum of such a series is dominated by the first term (the root). Therefore, the total time complexity is asymptotically bounded by the work done at the root.

Answer: $T(n) = \Theta(n)$

Q2 (GATE 2021 Style)

For constants $a \geq 1$ and $b > 1$, $T(n) = aT(n/b) + f(n)$. Under what conditions will $T(n) = \Theta(n \log n)$ hold?

Solution: According to the Master Theorem, $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ occurs in Case 2 when $f(n) = \Theta(n^{\log_b a} \log^{k} n)$.
For the result to be $\Theta(n \log n)$, we must match the powers of n and $\log n$. We must have $\log_b a = 1$ (which implies $a = b$) and $k = 0$. Substituting $k = 0$ and $\log_b a = 1$ into the condition gives $f(n) = \Theta(n)$.

Answer: $a = b$ and $f(n) = \Theta(n)$

Q3 (GATE 2024 Style)

An algorithm checks whether an array of size N is sorted by making a single pass and comparing only adjacent elements. What is the tightest asymptotic running time?

Solution: A single pass comparing adjacent elements loops from index 0 to $N-2$. In the worst case (e.g., the array is sorted, and we must check all pairs before confirming), it makes exactly $N-1$ comparisons. Thus, the maximum time taken is directly proportional to N.

Answer: $\Theta(N)$ or $O(N)$

Q4 (GATE 2015 Style)

Evaluate the asymptotic order of: $1^3 + 2^3 + 3^3 + ... + n^3$. Which of the following are true?
• $\Theta(n^{\spadesuit})$ **(Options: $\Theta(n^4)$, $\Theta(n^5)$, $O(n^5)$, $\Omega(n^3)$)**

Solution: The sum of the cubes of the first n natural numbers is given by the formula $[n(n+1)/2]^2 = (n^4 + 2n^3 + n^2)/4$.
The highest degree term is n^4, so the exact tight bound is $\Theta(n^4)$.
By the definitions of asymptotic notations:

- $\Theta(n^4)$ is **True**.
- $\Theta(n^5)$ is **False**.
- $O(n^5)$ is **True** (since n^4 is asymptotically smaller than or equal to n^5).
- $\Omega(n^3)$ is **True** (since n^4 is asymptotically greater than or equal to n^3).

Answer: $\Theta(n^4)$, $O(n^5)$, and $\Omega(n^3)$ are true.

Q5 (GATE 2015 Style)

Find the asymptotic value of `q`:

```
    int i,j,k,p,q=0;

    for(i=1; i<n; ++i) {

        p=0;

        for(j=n; j>1; j=j/2) ++p;

        for(k=1; k<p; k=k*2) ++q;

    }
```

Solution: Let's break down the loops:

- The outermost loop (variable i) runs from 1 to $n-1$, executing $\Theta(n)$ times.
- For each i, the first inner loop divides j by 2 starting from n. It executes $\log_2 n$ times, so $p = \lfloor \log_2 n \rfloor$.
- The second inner loop multiplies k by 2 starting from 1 until it reaches p. This executes $\log_2 p$ times.
- Substituting p, this innermost loop runs $\log_2(\log_2 n)$ times for each iteration of the outer loop.

Variable q is incremented in the innermost loop, so its final value is the product of the outer loop iterations and the inner loop iterations.

Answer: $q = \Theta(n \log(\log n))$

Q6 (Classic GATE)

Determine the complexity:

```
    for(int i=n; i>0; i/=2) {

        for(int j=0; j<i; j++) { /* constant work */ }

    }
```

Solution: The outer loop variable *i* takes on the values *n, n/2, n/4, ..., 1*.
The inner loop runs *i* times for each value of *i*.
Total work is proportional to the sum: $n + n/2 + n/4 + ... + 1 = n(1 + 1/2 + 1/4 + ...)$.
This is a geometric series that converges to $2n$.

Answer: $\Theta(n)$

Q7 (Classic GATE)

Determine the complexity:

```
    for(int i=1; i<=n; i++) {

        for(int j=1; j<=i; j++) { /* constant work */ }

    }
```

Solution: The outer loop runs from *1* to *n*. The inner loop runs exactly *i* times for each iteration.
Total iterations = $1 + 2 + 3 + ... + n = n(n+1)/2$. The dominant term is n^2.

Answer: $\Theta(n^2)$

Q8 (Recursion)

Determine the complexity:

```
    void fun(int n) {
```

```
        if(n<=1) return;

        fun(n/2);

        fun(n/2);

    }
```

Solution: The function makes two recursive calls with size $n/2$ and does constant work otherwise.
The recurrence relation is: $T(n) = 2T(n/2) + \Theta(1)$.
Using the Master Theorem with $a=2, b=2$, we calculate $n^{\log_2 2} = n^1 = n$. Since $f(n) = \Theta(1)$ is bounded above by $n^{1-\varepsilon}$ (Case 1), the complexity is dictated by the recursive calls.

Answer: $\Theta(n)$

Q9 (Fibonacci Pattern)

Determine the complexity:

```
    int fib(int n) {

        if(n<=1) return n;

        return fib(n-1) + fib(n-2);

    }
```

Solution: The recurrence relation is $T(n) = T(n-1) + T(n-2) + \Theta(1)$.
This forms a recursion tree where the number of nodes grows exponentially, mirroring the Fibonacci sequence itself. The tree has a maximum depth of n, leading to a loose upper bound of $O(2^{\spadesuit})$ (where $\spadesuit$ is n). A tighter analysis yields a base of the golden ratio, $\varphi \approx 1.618$.

Answer: $O(2^n)$ **or more tightly** $\Theta(1.618^n)$

Q10 (GATE Favorite)

Arrange the following in increasing order: $n^{3/2}$, $n \log n$, $n^{\log_2 n}$, 2^n

Solution: Let's evaluate the growth rates:

- $n \log n$ grows slower than any polynomial with a degree strictly greater than 1.
- $n^{3/2} = n^{1·&sup5;}$. Since $1.5 > 1$, $n \log n < n^{3/2}$.
- $n^{\log_2 n}$ can be rewritten as $2^{(\log_2 n)^2}$. Since $(\log_2 n)^2$ grows much faster than any constant, it eventually overtakes any polynomial, so $n^{3/2} < n^{\log_2 n}$.
- 2^n is a standard exponential. For large n, $n > (\log_2 n)^2$, so $n^{\log_2 n} < 2^n$.

Answer: $n \log n < n^{3/2} < n^{\log_2 n} < 2^n$

Additional PSU-Level Questions

Q11: Determine $T(n) = T(n-1) + 1$

Solution: The problem size decreases by 1 at each step, taking n steps to reach the base case. Constant work is done at each step.

Answer: $\Theta(n)$

Q12: Determine $T(n) = T(n-1) + n$

Solution: Expanding this yields $n + (n-1) + (n-2) + ... + 1$, which sums to $n(n+1)/2$.

Answer: $\Theta(n^2)$

Q13: Determine $T(n) = 2T(n-1) + 1$

Solution: This creates a full binary tree of height n. Summing the nodes yields $1 + 2 + 4 + ... + 2^{n-1} = 2^n - 1$.

Answer: $\Theta(2^n)$

Q14: Determine $T(n) = T(n/2) + 1$

Solution: The problem size is halved at each step until it reaches 1. The number of steps is $\log_2 n$. Constant work is done at each step (e.g., Binary Search).

Answer: $\Theta(\log n)$

Q15: Determine the complexity:

```
    for(int i=1; i<=n; i*=2) {

        for(int j=1; j<=n; j++) { /* constant work */ }
```

```
    }
```

Solution: The outer loop doubles i in each iteration, running $\log_2 n$ times.

For each outer iteration, the inner loop runs exactly n times.
The total complexity is the product of the two.

Answer: $\Theta(n \log n)$